

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Display all columns
pd.set_option('display.max_columns', None)

print("Libraries Imported Successfully!")
```

Libraries Imported Successfully!

```
df = pd.read_csv("/content/AB_NYC_2019.csv")

df.head()
```

	id	name	host_id	host_name	neighbourhood_group	neighbourhood
0	2539	Clean & quiet apt home by the park	2787	John	Brooklyn	Kensin
1	2595	Skylit Midtown Castle	2845	Jennifer	Manhattan	Midt
2	3647	THE VILLAGE OF HARLEM....NEW YORK !	4632	Elisabeth	Manhattan	Ha
3	3831	Cozy Entire Floor of Brownstone	4869	LisaRoxanne	Brooklyn	Clintor
4	5022	Entire Apt: Spacious Studio/Loft by central park	7192	Laura	Manhattan	East Ha

Next steps:

[Generate code with df](#)
[New interactive sheet](#)

```
print("Shape of Dataset:", df.shape)

print("\nColumn Names:")
print(df.columns)

print("\nDataset Information:")
df.info()
```

Shape of Dataset: (48895, 16)

Column Names:

```
Index(['id', 'name', 'host_id', 'host_name', 'neighbourhood_group',
      'neighbourhood', 'latitude', 'longitude', 'room_type', 'price',
      'minimum_nights', 'number_of_reviews', 'last_review',
      'reviews_per_month', 'calculated_host_listings_count',
      'availability_365'],
      dtype='object')
```

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 48895 entries, 0 to 48894

Data columns (total 16 columns):

#	Column	Non-Null Count	Dtype
0	id	48895 non-null	int64
1	name	48879 non-null	object
2	host_id	48895 non-null	int64
3	host_name	48874 non-null	object
4	neighbourhood_group	48895 non-null	object
5	neighbourhood	48895 non-null	object
6	latitude	48895 non-null	float64
7	longitude	48895 non-null	float64
8	room_type	48895 non-null	object
9	price	48895 non-null	int64
10	minimum_nights	48895 non-null	int64
11	number_of_reviews	48895 non-null	int64
12	last_review	38843 non-null	object
13	reviews_per_month	38843 non-null	float64
14	calculated_host_listings_count	48895 non-null	int64
15	availability_365	48895 non-null	int64

dtypes: float64(3), int64(7), object(6)

memory usage: 6.0+ MB

```
df.describe(include='all')
```

	id	name	host_id	host_name	neighbourhood_group	ne
count	4.889500e+04	48879	4.889500e+04	48874		48895
unique	NaN	47905	NaN	11452		5
top	NaN	Hillside Hotel	NaN	Michael		Manhattan
freq	NaN	18	NaN	417		21661
mean	1.901714e+07	NaN	6.762001e+07	NaN		NaN
std	1.098311e+07	NaN	7.861097e+07	NaN		NaN
min	2.539000e+03	NaN	2.438000e+03	NaN		NaN
25%	9.471945e+06	NaN	7.822033e+06	NaN		NaN
50%	1.967728e+07	NaN	3.079382e+07	NaN		NaN
75%	2.915218e+07	NaN	1.074344e+08	NaN		NaN
max	3.648724e+07	NaN	2.743213e+08	NaN		NaN

```
print(df.isnull().sum())
```

```
id                0
name              16
host_id           0
host_name         21
neighbourhood_group  0
neighbourhood     0
latitude          0
longitude         0
room_type         0
price            0
minimum_nights    0
number_of_reviews  0
last_review       10052
reviews_per_month  10052
calculated_host_listings_count  0
availability_365   0
dtype: int64
```

```
duplicates = df.duplicated().sum()
```

```
print("Duplicate Rows:", duplicates)
```

```
Duplicate Rows: 0
```

```
df = df.drop_duplicates()
```

```
print("New Shape:", df.shape)
```

```
New Shape: (48895, 16)
```

```
# Fill missing reviews_per_month with 0
df['reviews_per_month'] = df['reviews_per_month'].fillna(0)
```

```
# Convert last_review to datetime
df['last_review'] = pd.to_datetime(df['last_review'])
```

```
print(df.isnull().sum())
```

```
id                0
name              16
host_id           0
host_name         21
neighbourhood_group  0
neighbourhood     0
latitude          0
longitude         0
room_type         0
price            0
minimum_nights    0
number_of_reviews  0
last_review       10052
reviews_per_month  0
calculated_host_listings_count  0
```

```
availability_365      0
dtype: int64
```

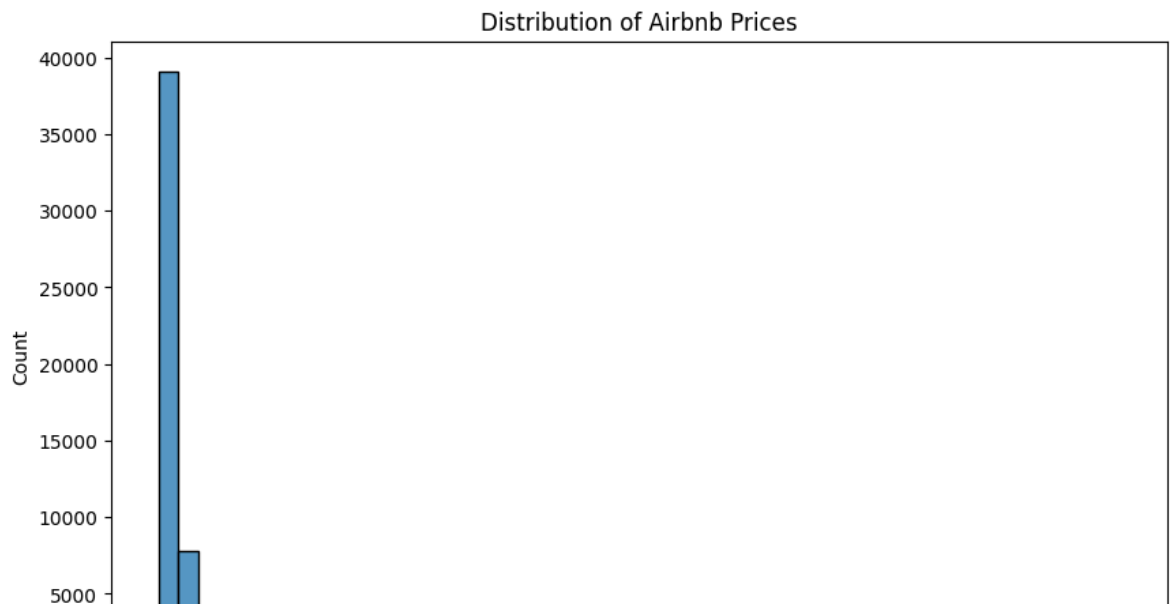
```
df.dtypes
```

```
      0
id      int64
name    object
host_id  int64
host_name object
neighbourhood_group object
neighbourhood object
latitude float64
longitude float64
room_type object
price    int64
minimum_nights int64
number_of_reviews int64
last_review  datetime64[ns]
reviews_per_month float64
calculated_host_listings_count int64
availability_365 int64
```

```
dtype: object
```

```
plt.figure(figsize=(10,6))
sns.histplot(df['price'], bins=50)

plt.title("Distribution of Airbnb Prices")
plt.xlabel("Price")
plt.ylabel("Count")
plt.show()
```



```
df = df[df['price'] < 600]
```

```
print(df.shape)
```

```
(48036, 16)
```

```
plt.figure(figsize=(10,6))
```

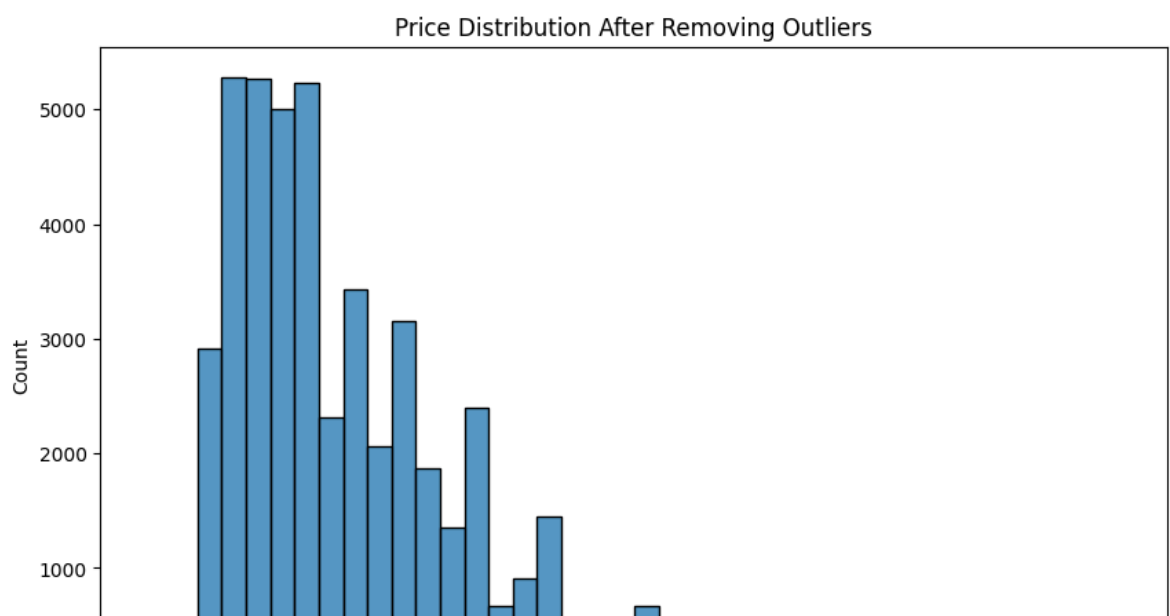
```
sns.histplot(df['price'], bins=40)
```

```
plt.title("Price Distribution After Removing Outliers")
```

```
plt.xlabel("Price")
```

```
plt.ylabel("Count")
```

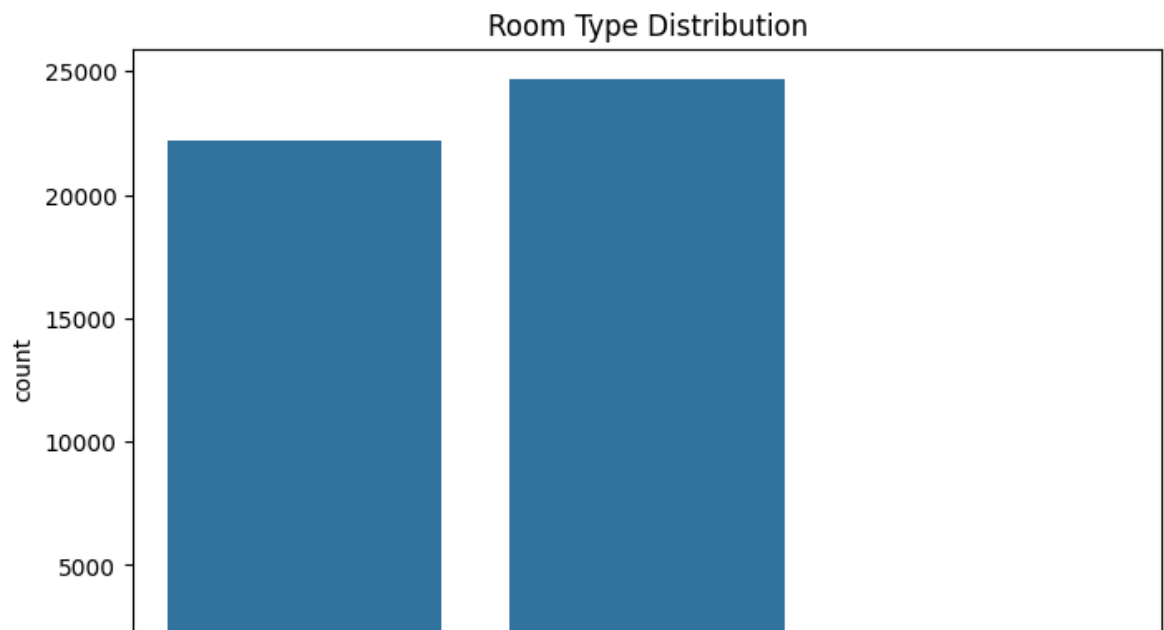
```
plt.show()
```



```
plt.figure(figsize=(8,5))
```

```
sns.countplot(x='room_type', data=df)
```

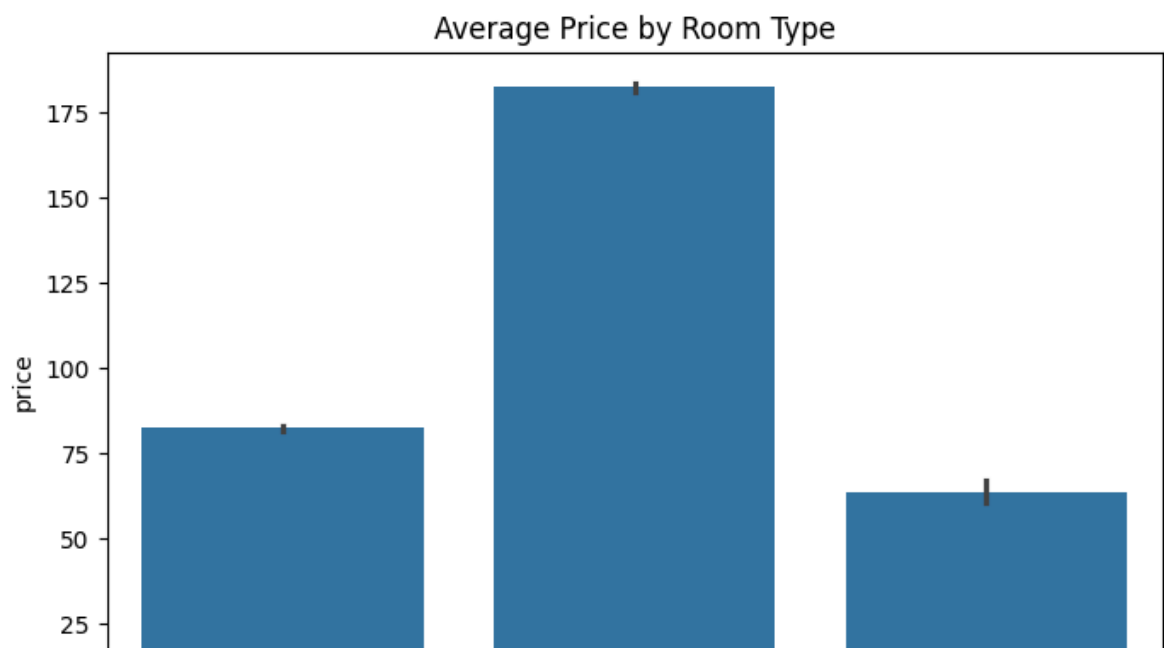
```
plt.title("Room Type Distribution")
plt.show()
```



```
plt.figure(figsize=(8,5))

sns.barplot(
    x='room_type',
    y='price',
    data=df,
    estimator=np.mean
)

plt.title("Average Price by Room Type")
plt.show()
```



```
plt.figure(figsize=(10,6))

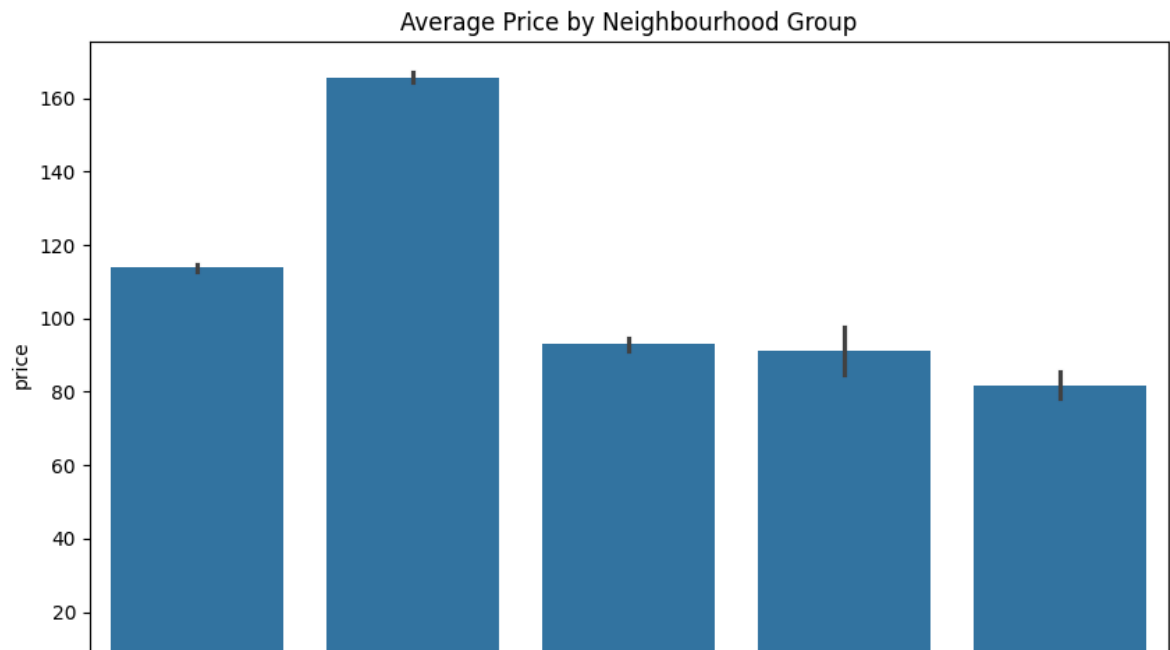
sns.barplot(
```

```

    x='neighbourhood_group',
    y='price',
    data=df,
    estimator=np.mean
)

plt.title("Average Price by Neighbourhood Group")
plt.xticks(rotation=20)
plt.show()

```



```

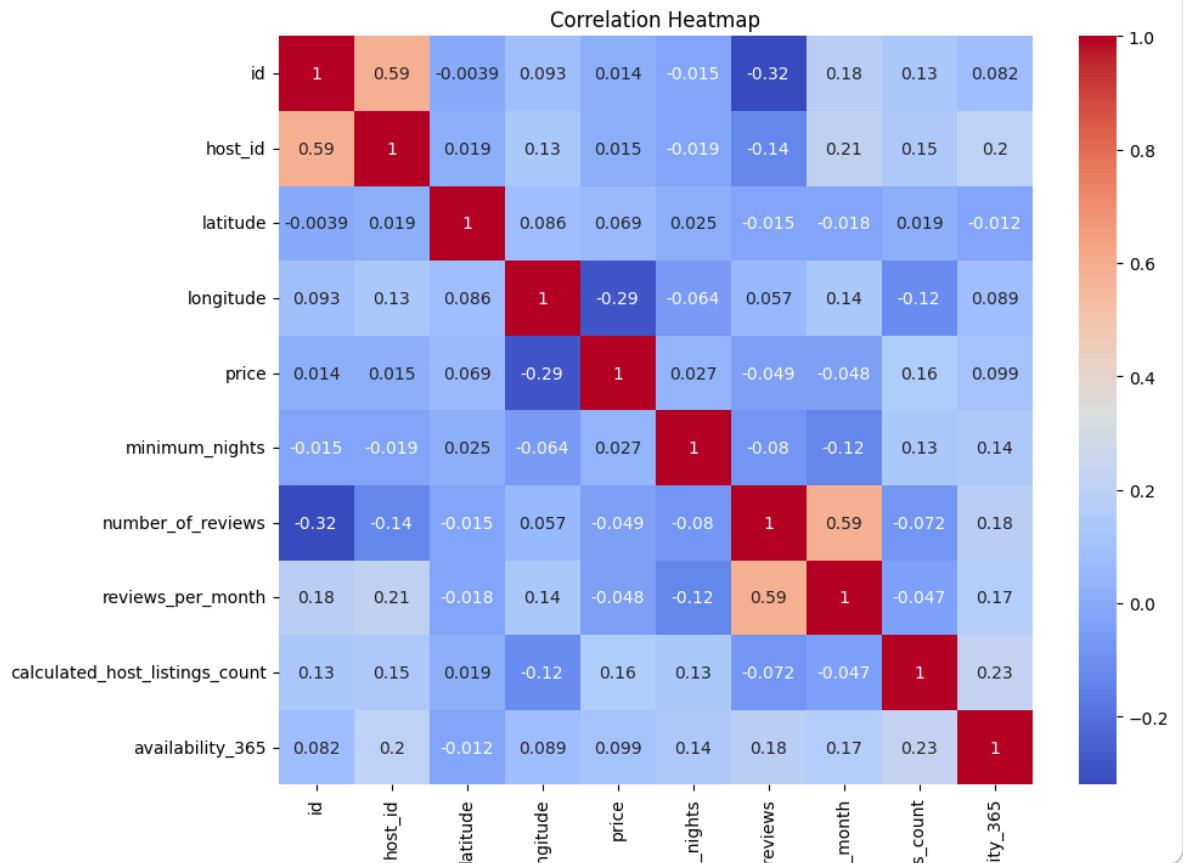
numeric_df = df.select_dtypes(include=np.number)

plt.figure(figsize=(10,8))

sns.heatmap(
    numeric_df.corr(),
    annot=True,
    cmap='coolwarm'
)

plt.title("Correlation Heatmap")
plt.show()

```



```

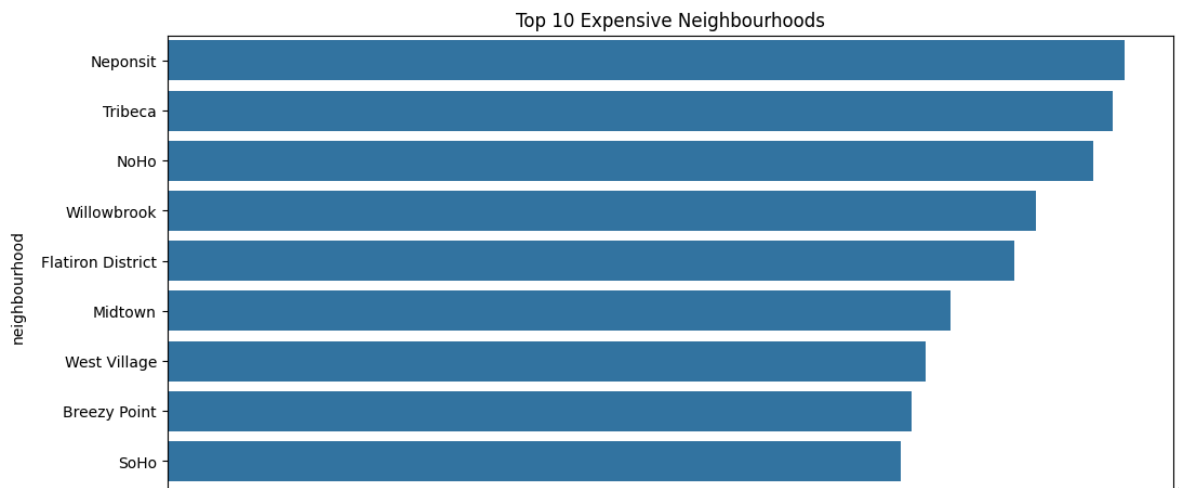
top = (
    df.groupby('neighbourhood')['price']
      .mean()
      .sort_values(ascending=False)
      .head(10)
)

plt.figure(figsize=(12,6))

sns.barplot(
    x=top.values,
    y=top.index
)

plt.title("Top 10 Expensive Neighbourhoods")
plt.xlabel("Average Price")
plt.show()

```

```
df.to_csv("cleaned_airbnb.csv", index=False)
```

```
print("Cleaned dataset saved successfully!")
```

Cleaned dataset saved successfully!

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
```

```
X = df[['neighbourhood_group',
        'room_type',
        'minimum_nights',
        'number_of_reviews',
        'reviews_per_month',
        'calculated_host_listings_count',
        'availability_365']]
```

```
y = df['price']
```

```
X.head()
```

	neighbourhood_group	room_type	minimum_nights	number_of_reviews	reviews
0	Brooklyn	Private room	1	9	
1	Manhattan	Entire home/apt	1	45	
2	Manhattan	Private room	3	0	
3	Brooklyn	Entire home/apt	1	270	
4	Manhattan	Entire home/apt	10	9	

Next steps:

[Generate code with X](#)

[New interactive sheet](#)

```
categorical_features = ['neighbourhood_group', 'room_type']

numerical_features = [
    'minimum_nights',
    'number_of_reviews',
    'reviews_per_month',
    'calculated_host_listings_count',
    'availability_365'
]
```

```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median'))
])

categorical_transformer = Pipeline(steps=[
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numerical_features),
        ('cat', categorical_transformer, categorical_features)
    ]
)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
print(X_train.shape)
print(X_test.shape)
```

```
(38428, 7)
(9608, 7)
```

```
lr_model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', LinearRegression())
])

lr_model.fit(X_train, y_train)

print("Linear Regression Model Trained Successfully!")
```

```
Linear Regression Model Trained Successfully!
```

```
y_pred_lr = lr_model.predict(X_test)
```

```
mae = mean_absolute_error(y_test, y_pred_lr)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_lr))
r2 = r2_score(y_test, y_pred_lr)

print("Linear Regression Results")
print("-----")
print("MAE :", round(mae,2))
print("RMSE:", round(rmse,2))
print("R2 Score:", round(r2,3))
```

```
Linear Regression Results
-----
MAE : 48.62
RMSE: 71.98
R2 Score: 0.381
```

```
rf_model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', RandomForestRegressor(
        n_estimators=100,
        random_state=42
    ))
])

rf_model.fit(X_train, y_train)

print("Random Forest Model Trained Successfully!")
```

```
Random Forest Model Trained Successfully!
```

```
y_pred_rf = rf_model.predict(X_test)
```

```

mae = mean_absolute_error(y_test, y_pred_rf)
rmse = np.sqrt(mean_squared_error(y_test, y_pred_rf))
r2 = r2_score(y_test, y_pred_rf)

print("Random Forest Results")
print("-----")
print("MAE :", round(mae,2))
print("RMSE:", round(rmse,2))
print("R² Score:", round(r2,3))

```

```

Random Forest Results
-----
MAE : 47.47
RMSE: 71.65
R² Score: 0.387

```

```

comparison = pd.DataFrame({
    "Actual Price": y_test.values,
    "Predicted Price": y_pred_rf
})

comparison.head(10)

```

	Actual Price	Predicted Price	
0	40	70.50	
1	104	145.68	
2	196	114.09	
3	120	105.15	
4	90	68.93	
5	210	174.19	
6	55	105.12	
7	75	65.15	
8	189	276.68	
9	100	74.84	

Next steps:

[Generate code with comparison](#)

[New interactive sheet](#)

```

rf = rf_model.named_steps['model']

feature_names = (
    numerical_features +
    list(rf_model.named_steps['preprocessor']
        .named_transformers_['cat']
        .named_steps['encoder']
        .get_feature_names_out(categorical_features))
)

```

```
importance = pd.DataFrame({
    "Feature": feature_names,
    "Importance": rf.feature_importances_
})

importance = importance.sort_values(by="Importance", ascending=False)
```

	Feature	Importance
10	room_type_Entire home/apt	0.333552
4	availability_365	0.179638
2	reviews_per_month	0.152990
1	number_of_reviews	0.111519
0	minimum_nights	0.089756
7	neighbourhood_group_Manhattan	0.060731
3	calculated_host_listings_count	0.058292